

UT2A4



Frontend



Backend

Github: <https://github.com/Rayan15100/UT2A4-Rayan-CRUD>

Actividad Pendiente UT2A4
Rayan Naim El Bachiri

FRONTEND.....	3
BACKEND.....	4
PRUEBA:.....	6

FRONTEND

Importamos el frontend de la anterior actividad desde github:

```
PS C:\Users\alumnado\Desktop\ProyectosReact> git clone -b redux https://github.com/Rayan15100/frontend.git
Cloning into 'frontend'...
remote: Enumerating objects: 49, done.
remote: Counting objects: 100% (49/49), done.
remote: Compressing objects: 100% (34/34), done.
remote: Total 49 (delta 11), reused 48 (delta 11), pack-reused 0 (from 0)
Receiving objects: 100% (49/49), 53.64 KiB | 446.00 KiB/s, done.
Resolving deltas: 100% (11/11), done.
PS C:\Users\alumnado\Desktop\ProyectosReact> cd frontend
PS C:\Users\alumnado\Desktop\ProyectosReact\frontend> npm install

added 294 packages, and audited 295 packages in 35s

58 packages are looking for funding
  run `npm fund` for details

12 vulnerabilities (6 moderate, 6 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
npm notice
npm notice New minor version of npm available! 11.6.0 -> 11.13.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.13.0
npm notice To update run: npm install -g npm@11.13.0
npm notice
PS C:\Users\alumnado\Desktop\ProyectosReact\frontend>
```

Para organizar mejor el código, se cambió la lógica a la carpeta src/components/

Menu.tsx:

- Se implementó la barra de navegación superior (AppBar) con Material UI.
- Se incluyó un menú lateral (Drawer) para navegar entre secciones.
- Se añadió la protección de rutas: un useEffect que verifica si el usuario está autenticado en Redux y si no lo está lo expulsa automáticamente al Login.

Dashboard.tsx:

- Es donde se define el CRUD.
- Contiene el formulario para capturar datos (Nombre, Marca, Tipo, Precio).
- Gestiona la Tabla de datos.
- Implementa las funciones con el servidor (fetch) para cargar la lista, insertar nuevos elementos y borrar registros por ID.

Se actualizaron las páginas existentes para conectarlas con el backend y con los nuevos componentes:

Login.tsx: Se eliminaron las credenciales fijas (bduser/bdpasswd). Ahora usa fetch para consultar el endpoint /login de tu servidor Node.js y validar contra la tabla usuarios de MySQL. (En este caso es "Patricia" "123456789").

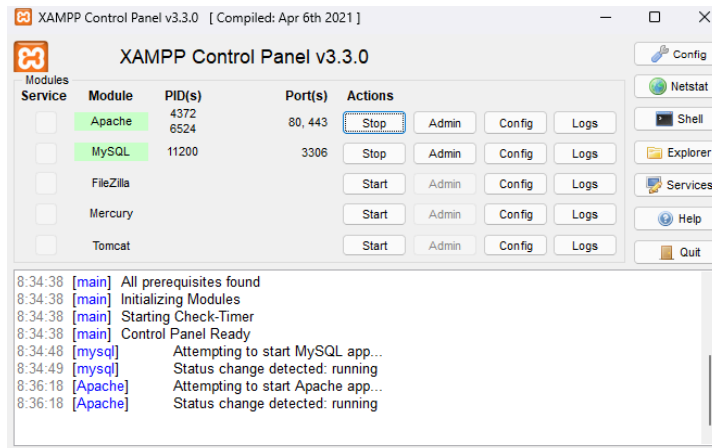
Homes.tsx: Pasó de ser una página con texto normal a un contenedor. Ahora solo invoca a <Menu /> y <Dashboard />.

App.tsx: Me aseguré de la correcta definición de rutas para que /home cargue el nuevo componente.

BACKEND

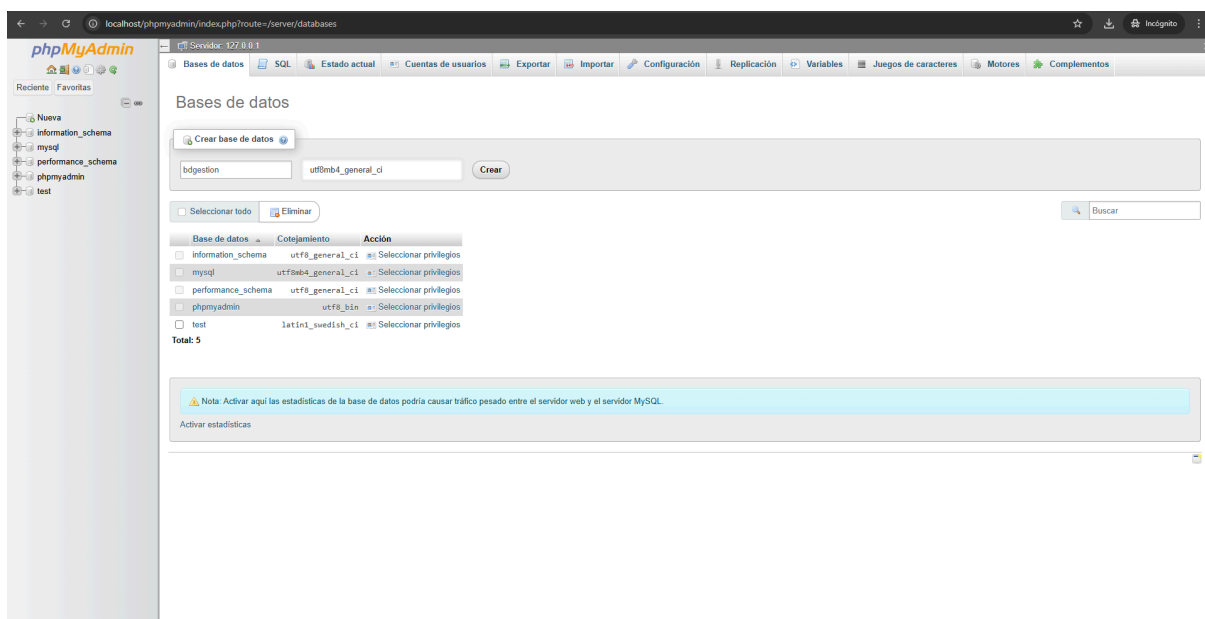
Para empezar instalaremos Xampp mediante el enlace facilitado en el campus.

Una vez instalado correctamente iniciaremos APACHE y MYSQL.

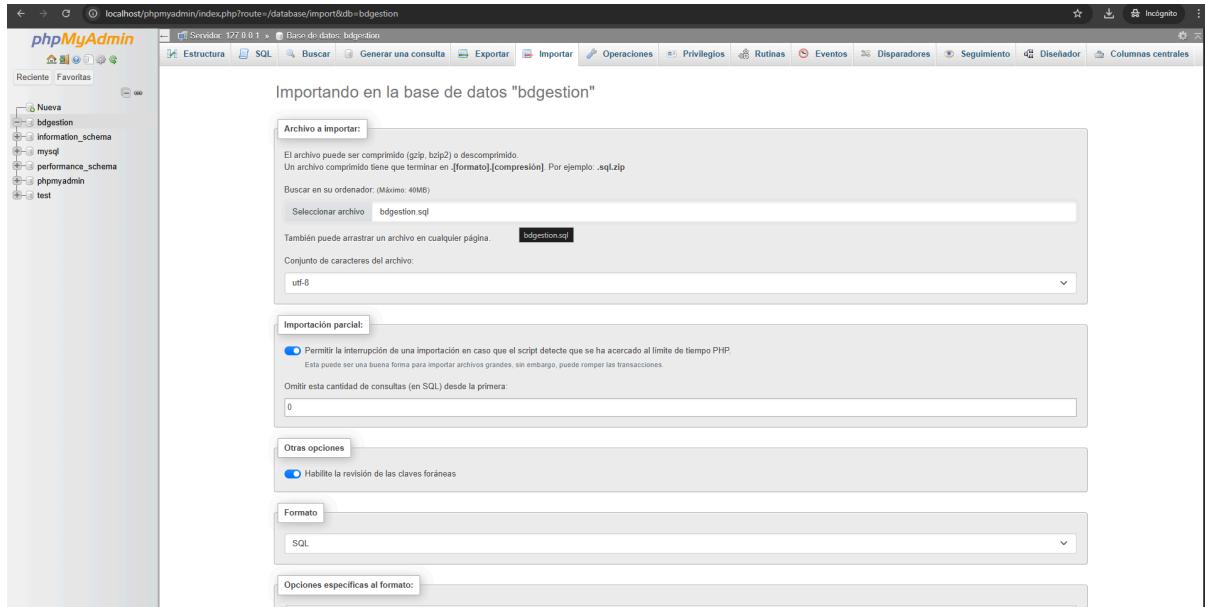


Una vez iniciado buscaremos en nuestro buscador la siguiente conexión:
<http://localhost/phpmyadmin/>

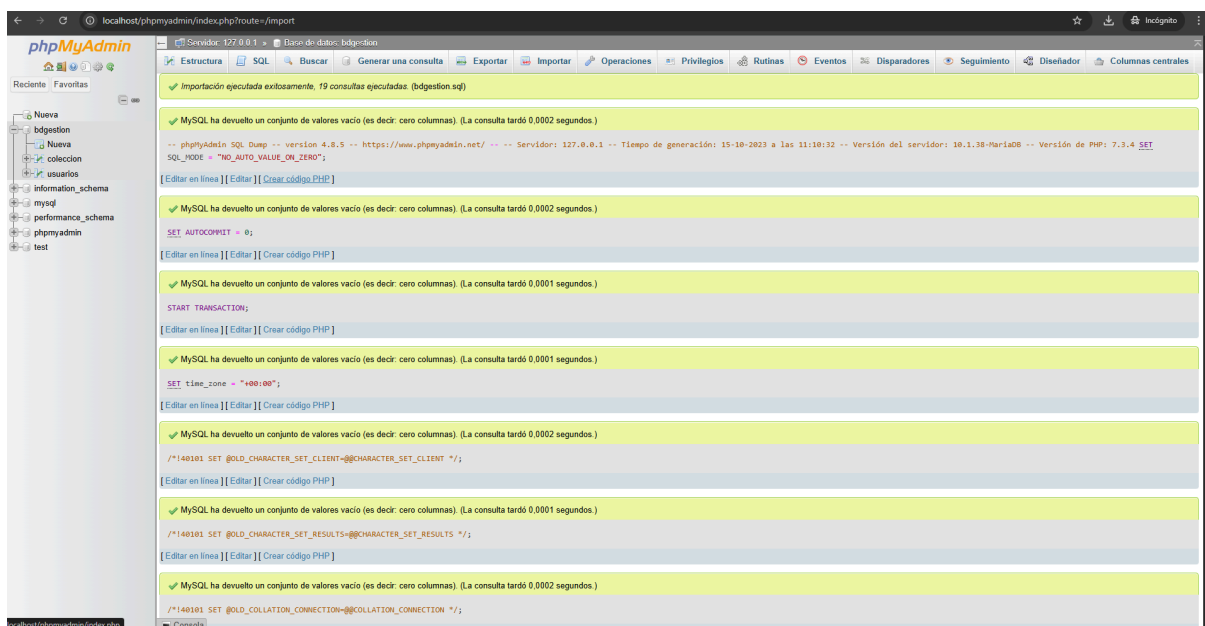
Una vez dentro crearemos una nueva base de datos llamada **“bdgestion”**



Una vez creada importamos la base de datos que nos han facilitado en el campus.



Una vez importada se nos crearan 2 tablas Usuarios y Colección:



Una vez realizado esto, creamos una carpeta Backend en la raíz principal a la altura del Frontend

Creamos la carpeta Services.

E incluimos los archivos de configuración que nos han proporcionado como son: db.js, config.js, helper.js, index.js.

Creamos Items.js

Este archivo es el que se comunica con la tabla colección en el MySQL con tres funciones:

- `getData()`: Ejecuta un `SELECT * FROM coleccion` para leer todos los objetos guardados y enviarlos al frontend.
- `insertData(req)`: Ejecuta un `INSERT INTO` recogiendo las variables que le manda el frontend (nombre, marca, tipo, precio) por la URL.
- `deleteData(req)`: Ejecuta un `DELETE FROM` buscando por el id exacto del objeto que el usuario quiere eliminar.

Para que el frontend pudiera usar las funciones de `items.js`, tuve que declararlo en el archivo principal `index.js`. Añadí tres nuevos endpoints (rutas):

- `/getItems`: Llama a `items.getData()` y devuelve la lista de la colección al Dashboard.
- `/addItem`: Llama a `items.insertData()`. Es el que recibe los datos cuando pulsas "+ INSERTAR DATOS".
- `/deleteItem`: Llama a `items.deleteData()`. Se activa cuando pulsas la papelera roja en la tabla.

PRUEBA:

